Lab 5 – Rapport individuel – Patient Management

Nov 2, 2026 · @Thanh

J'ai pris en charge la fonctionnalité **Patient Management** : la route `/api/patients` (GET et POST), ses tests automatisés et ses requêtes Postman, livrées via le ticket Jira DEVOPS-11 et la PR #5, mergée dans `main` avec une CI verte.

Élément	Valeur
Étudiant	Thanh Son Potherat (GitHub : Thanh-Poth)
Groupe	Luca Perkins (LP), Gustave Perdigeon (GP), Zixuan Qian (ZQ), Thanh Son Potherat (TP)
Fonctionnalité	Patient Management – `GET /api/patients`, `POST /api/patients`
Ticket Jira	DEVOPS-11 (story), epic DEVOPS-5 API Core Development
Dépôt GitHub (public)	github.com/Thanh-Poth/Software_Engineering_Lab_4
Pull request	#5 – feature/DEVOPS-11-patients-route

1. Contexte et objectifs

Le Lab 5 prolonge le projet du Lab 4 (API Express, CI GitHub Actions, board Jira) : chaque étudiant implémente et teste une route REST de l'application Healthcare. Le travail se déroule dans le même dépôt, avec la même CI et le même board Jira.

Objectifs couverts par ce rapport :

- consolider les bases JavaScript utilisées en NodeJS (variables, tableaux, objets, fonctions, modules, erreurs, async/await) ;
- implémenter la route Patient Management `/api/patients` ;
- la tester automatiquement avec Vitest et Supertest, puis manuellement avec Postman ;
- livrer le tout selon le workflow DevOps : ticket Jira, branche, commit, pull request, CI.

2. Pratique JavaScript

Les sept exercices sont faits dans `js-practice.js`, sauf la modularisation (2.5) qui est répartie entre `hospitalData.js` et `main.js`. Les deux scripts s'exécutent sans erreur avec `node js-practice.js` et `node main.js`.

Exercice	Ce qui est fait	Sortie console obtenue
2.1 Variables	`const` pour le nom et le nombre de médecins, `let` pour `isOpen`, message différent si l'hôpital est fermé	Welcome to Central Hospital! We have 12 doctors currently available. / Central Hospital is currently closed.
2.2 Tableaux et boucles	Tableau de 5 médecins, boucle `for...of` avec `entries()`, `push()`, fonction `findDoctor(name)`	Doctor #1: Dr. Lee … Doctor #5 / Doctors count: 6 / Doctor found / Doctor not found
2.3 Objets imbriqués	Objet `patient` avec un tableau `conditions` et un objet `doctor`, ajout dynamique de « anxiety »	Alice Martin is treated by Dr. Lee (Cardiology).
2.4 Filtrage	`filterByAge(minAge)` avec `filter()`, `addPatient(name, age)`	Older than 30: Alice (34), John (45)
2.5 Modules	`hospitalData.js` exporte `doctors` et `addDoctor()`, `main.js` les importe	Updated doctors: ['Dr. Lee', 'Dr. Khan', 'Dr. Martin']
2.6 Erreurs	`createAppointment()` lève une erreur si un champ manque, capturée par `try/catch`	Error: Missing required fields
2.7 Asynchrone	`getHospitalData()` renvoie une `Promise` résolue après 1 000 ms, lue avec `await`	Data loaded: { name: 'Central Hospital', uptime: 1234 }

Ces notions servent directement dans la route : objets JSON, `filter` / `push` sur un tableau, validation des champs et modules ES (`import` / `export`).

Le projet déclare `"type": "module"` dans `package.json`, ce qui autorise la syntaxe `import` / `export` des modules ES dans NodeJS.

3. Route Patient Management : `/api/patients`

La route expose deux endpoints dans `src/routes/auto/patients.route.js`. `GET` renvoie la liste des patients (200), `POST` en crée un (201), ou répond 400 si les données sont invalides.

```
import { Router } from 'express'

const router = Router()

const patients = [
  { id: 1, name: 'Alice', age: 30 },
  { id: 2, name: 'John', age: 45 }
]

router.get('/api/patients', (_req, res) => res.status(200).json(patients))

router.post('/api/patients', (req, res) => {
  const { name, age } = req.body
  if (typeof name !== 'string' || !name.trim() || !Number.isInteger(age) ||
age <= 0) {
    return res.status(400).json({ error: 'Missing or invalid fields (name:
string, age: positive integer)' })
  }
  const newPatient = { id: patients.length + 1, name: name.trim(), age }
  patients.push(newPatient)
  res.status(201).json(newPatient)
})

export default router
```



Figure 8 – `src/routes/auto/patients.route.js` dans VS Code : routes `GET` et `POST /api/patients` avec validation du corps de la requête.

Méthode	Corps attendu	Réponse
`GET /api/patients`	aucun	200 + tableau de patients
`POST /api/patients`	`{ "name": "Marie", "age": 29 }`	201 + `{ "id": 3, "name": "Marie", "age": 29 }`
`POST /api/patients` (invalide)	`{ "name": "NoAge" }`	400 + `{ "error": "Missing or invalid fields …" }`

Choix d'implémentation :

- **Validation plus stricte que l'exemple du sujet.** Le sujet teste seulement `!name || !age`, ce qui accepte `age: "old"` ou `age: -3`. Ici, le nom doit être une chaîne non vide et l'âge un entier strictement positif.
- **Parsing JSON.** J'ai ajouté `app.use(express.json())` dans `src/app.js`, sans quoi `req.body` est vide sur les POST. Ce middleware initialise toujours `req.body` à `{}`, donc aucun garde supplémentaire n'est nécessaire.
- **Montage automatique.** Le fichier est dans `src/routes/auto/` et finit par `.route.js` : `app.js` le monte sans modification, ce qui évite les conflits de merge avec les routes des autres membres.

- **Stockage en mémoire.** Les patients sont dans un tableau : les ajouts sont perdus au redémarrage du serveur, ce qui suffit pour le lab (pas de base de données demandée).

4. Tests automatisés (Vitest + Supertest)

Les 10 tests de `test/patients.test.js` passent et couvrent `patients.route.js` à 100 % (lignes, branches, fonctions). Supertest envoie de vraies requêtes HTTP à l'application Express, sans démarrer de serveur.

Le sujet cite Jest, mais le projet utilise Vitest, dont la syntaxe (`describe`, `it`, `expect`) est identique.

Test	Requête	Résultat attendu
Liste des patients	`GET /api/patients`	200, un tableau dont les éléments ont un `name`
Création	`POST { name: 'Marie', age: 29 }`	201, un `id`, les champs renvoyés à l'identique
Persistance	`POST` Zoe, puis `GET`	Zoe apparaît dans la liste
Corps invalides (6 cas, `it.each`)	`{}`, nom seul, âge seul, nom vide, âge négatif, âge texte	400 + un champ `error`
Aucun corps	`POST` sans body	400

Extrait (sur le modèle de la section 3.3 du sujet) :

```
it.each([
  [{}],
  [{ name: 'NoAge' }],
  [{ age: 20 }],
  [{ name: '  ', age: 20 }],
  [{ name: 'Bad', age: -3 }],
  [{ name: 'Bad', age: 'old' }]
])('returns 400 for invalid body %j', async (body) => {
  const res = await request(app).post('/api/patients').send(body)
  expect(res.status).toBe(400)
  expect(res.body).toHaveProperty('error')
})
```

Software_Engineering_Lab_4 > test > JS patients.test.js > ...

```
/**
 * Integration tests for /api/patients.
 */
import request from 'supertest'
import app from '../src/app.js'
import { describe, it, expect } from 'vitest'

describe('GET /api/patients', () => {
  it('returns an array of patients', async () => {
    const res = await request(app).get('/api/patients')
    expect(res.status).toBe(200)
    expect(Array.isArray(res.body)).toBe(true)
    expect(res.body[0]).toHaveProperty('name')
  })
})

describe('POST /api/patients', () => {
  it('creates a patient and returns 201 with an id', async () => {
    const res = await request(app).post('/api/patients').send({ name: 'Marie', age: 29 })
    expect(res.status).toBe(201)
    expect(res.body).toHaveProperty('id')
    expect(res.body).toMatchObject({ name: 'Marie', age: 29 })
  })

  it('makes the new patient visible in GET', async () => {
    await request(app).post('/api/patients').send({ name: 'Zoe', age: 51 })
    const res = await request(app).get('/api/patients')
    expect(res.body.some(p => p.name === 'Zoe')).toBe(true)
  })

  it.each([
    [{}],
    [{ name: 'NoAge' }],
    [{ age: 20 }],
    [{ name: '   ', age: 20 }],
    [{ name: 'Bad', age: -3 }],
    [{ name: 'Bad', age: 'old' }]
  ])('returns 400 for invalid body %j', async (body) => {
    const res = await request(app).post('/api/patients').send(body)
    expect(res.status).toBe(400)
    expect(res.body).toHaveProperty('error')
  })

  it('returns 400 when no body is sent', async () => {
    const res = await request(app).post('/api/patients')
    expect(res.status).toBe(400)
  })
})

```

Figure 9 – `test/patients.test.js` dans VS Code : tests d'intégration Vitest + Supertest de `GET` et `POST /api/patients`, dont le cas `it.each` pour les corps invalides.

Couverture du projet entier mesurée en local (`npm test -- --coverage` , 28 tests, route doctors de l'équipe incluse) :

Indicateur	Couverture	Seuil CI
Lignes	95,45 % (63/66)	80 %
Instructions	94,66 % (71/75)	80 %
Fonctions	86,66 % (13/15)	80 %
Branches	83,33 % (25/30)	70 %

Les routes (`src/routes/auto`) et les utilitaires (`src/utils`) sont couverts à 100 %. Les seules branches non couvertes sont dans `src/app.js` et `src/index.js` , le code de démarrage commun. Sur la branche de finition poussée sur GitHub (route appointments de l'équipe mergée, route doctors pas encore), 45 tests passent et la couverture des branches est de 80 % : les quatre indicateurs atteignent les 80 % exigés par le sujet (section 6).

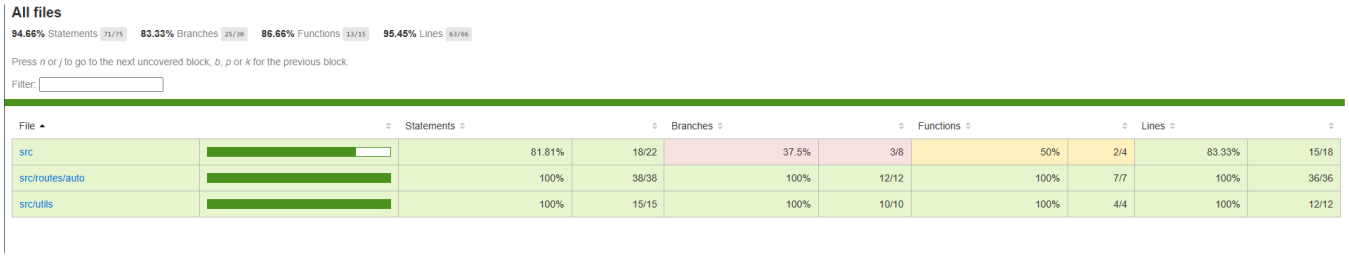


Figure 7 – Rapport `coverage/index.html` : 94,66 % des instructions, 83,33 % des branches, 86,66 % des fonctions, 95,45 % des lignes.

5. Tests avec Postman

La collection **Healthcare API Tests** passe entièrement au Runner : 8 assertions sur 8 réussies, 0 erreur, temps de réponse moyen de 15 ms. Les requêtes sont envoyées depuis l'application de bureau Postman, les corps POST en Body → raw → JSON. Le serveur tourne en local avec `npm run dev` sur `http://localhost:3000` , défini dans la variable de collection `{{baseUrl}}` .

Requête	Corps envoyé	Statut obtenu	Assertions `pm.test`
`GET /api/patients`	—	200 OK (19 ms)	statut 200, réponse = tableau
`POST /api/patients`	`{ "name": "Marie", "age": 29 }`	201 Created (29 ms)	statut 201, présence d'un `id`
`POST /api/patients` (invalide)	`{ "name": "NoAge" }`	400 Bad Request (6 ms)	statut 400, présence d'un `error`

Requête	Corps envoyé	Statut obtenu	Assertions `pm.test`
`GET /api/doctors`	—	200 OK	statut 200, réponse = tableau

Les trois premières requêtes sont les miennes (Patient Management). La collection exportée contient aussi les requêtes doctors et appointments des autres membres : les quatre requêtes demandées en section 4.2 (GET /api/doctors, POST /api/doctors, GET /api/patients, POST /api/appointments) y figurent, soit 8 requêtes au total.

La collection d'équipe finale (8 requêtes : patients, doctors, appointments) a aussi été exécutée en ligne de commande avec Newman, le runner CLI de Postman : 16 assertions sur 16 réussies, 0 échec, 14 ms de temps de réponse moyen.

Assertions ajoutées dans l'onglet Tests (Scripts → Post-response) de `POST /api/patients` :

```
pm.test("Status code is 201", function () {
  pm.response.to.have.status(201);
});
pm.test("Patient has an id", function () {
  pm.expect(pm.response.json()).to.have.property("id");
});
```

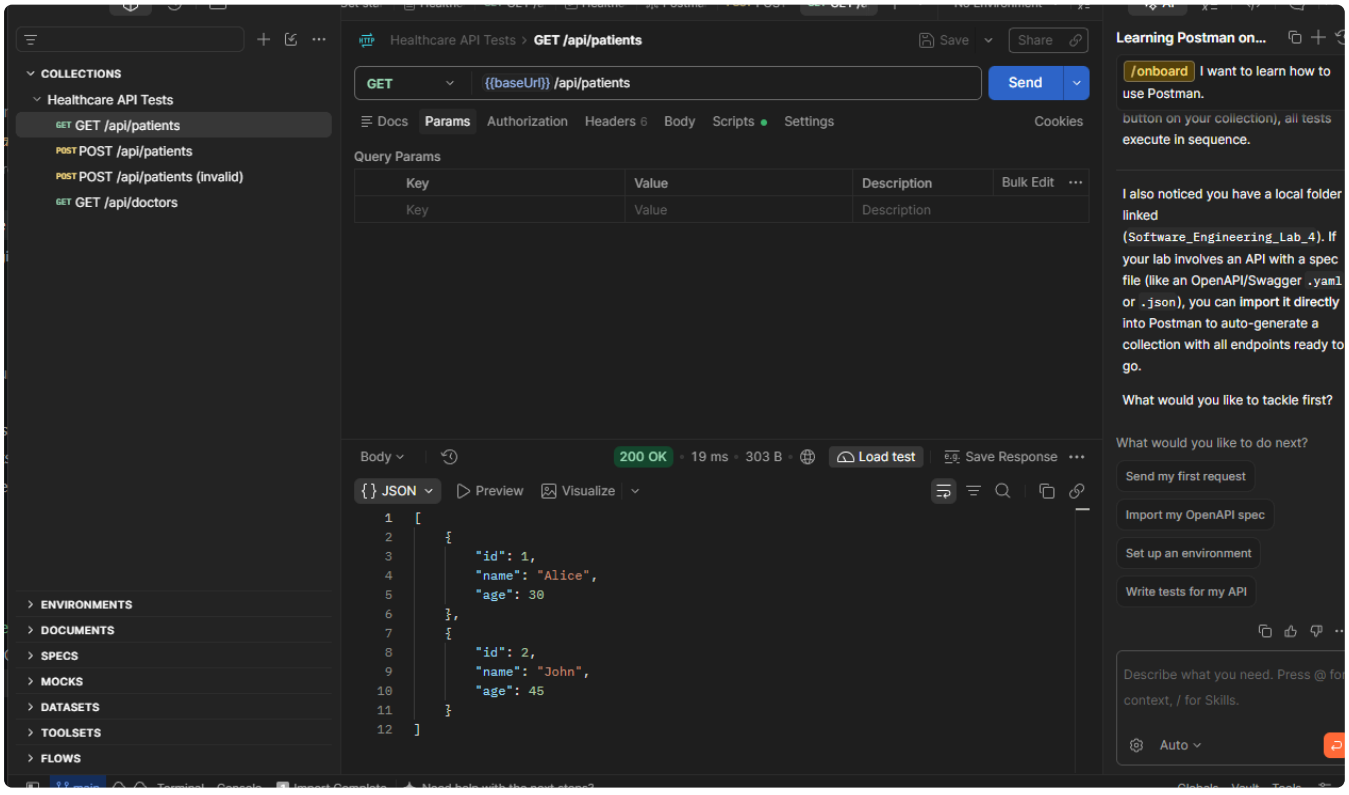


Figure 1 – `GET /api/patients` : 200 OK en 19 ms, les deux patients initiaux.

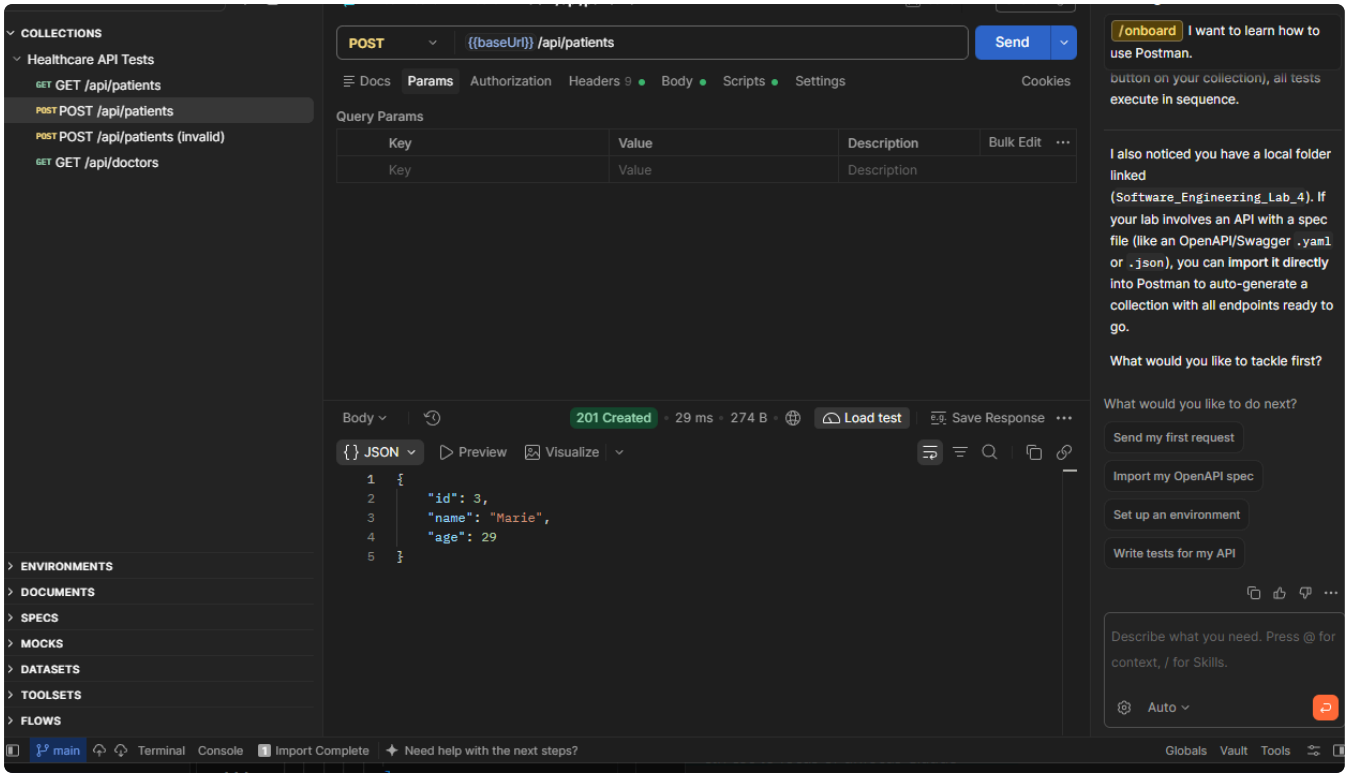


Figure 2 – `POST /api/patients` : 201 Created en 29 ms, patient Marie créé avec l'id 3.

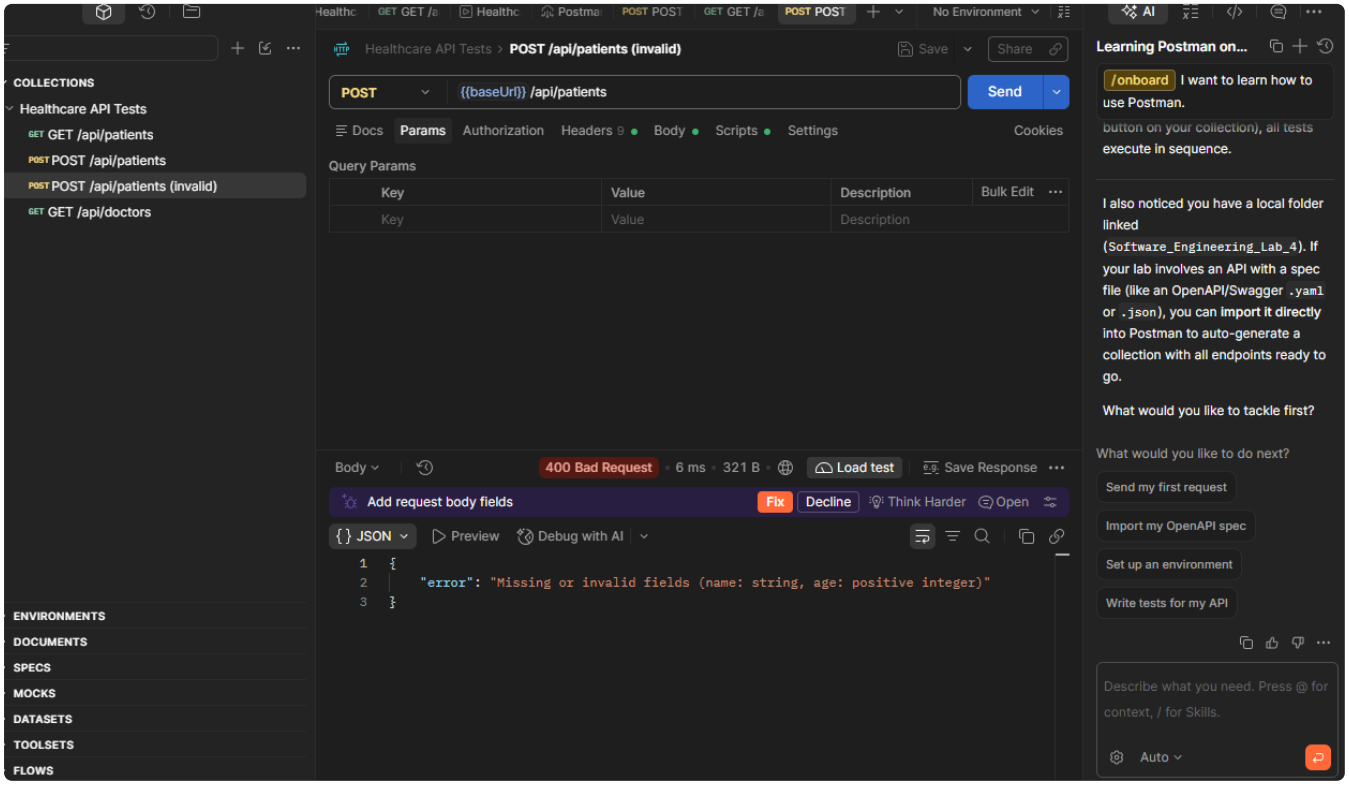


Figure 3 – `POST /api/patients` sans `age` : 400 Bad Request en 6 ms, avec le message d'erreur de validation.

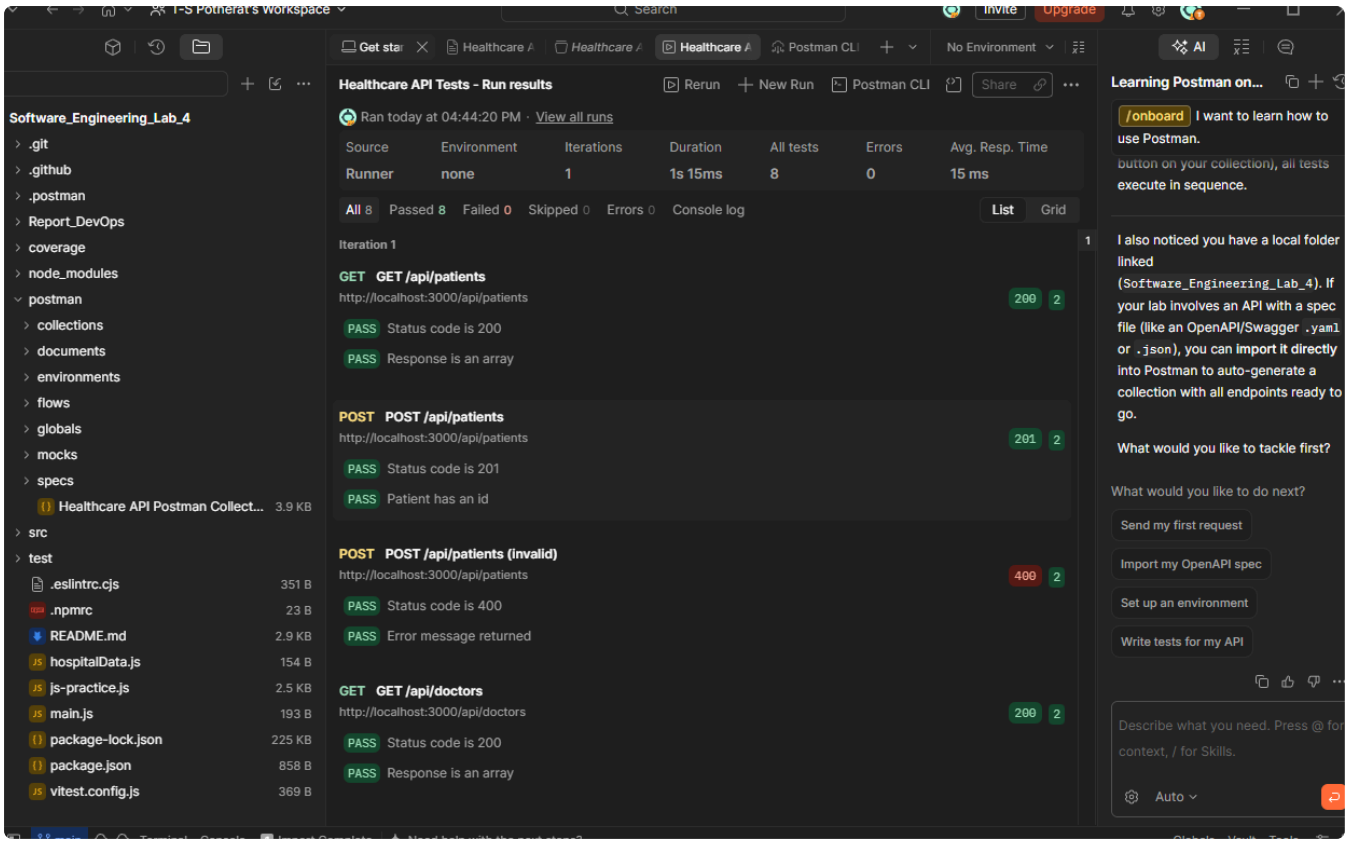


Figure 4 – Runner de la collection : 4 requêtes, 8 assertions réussies, 0 échec, 15 ms de moyenne.

La collection est exportée au format v2.1 sous `postman/Healthcare_API_Postman_Collection.json` dans le dépôt (section 4.6).

6. Workflow DevOps et intégration continue

La route a suivi tout le cycle du sujet (section 5) et la CI GitHub Actions est passée au vert sur la PR comme sur `main` après le merge, le 2 novembre 2026.

User story DEVOPS-11 : en tant que membre du personnel hospitalier, je veux consulter et ajouter des patients via l'API, afin de gérer la liste des patients.

Critères d'acceptation, tous validés :

- ~~`GET /api/patients` renvoie 200 et un tableau de patients.~~
- ~~`POST /api/patients` crée un patient (`name` , `age`) et renvoie 201 avec l'id généré.~~
- ~~`POST` renvoie 400 si les champs sont manquants ou invalides.~~
- ~~Les nouveaux tests passent et les seuils de couverture de la CI sont respectés.~~
- ~~La branche, les commits et la PR incluent la clé DEVOPS-11.~~

1. **Jira** : story DEVOPS-11 « Patients Management », assignée à T-S Potherat, rattachée à l'epic DEVOPS-5 API Core Development, passée par To Do → In Progress → Review → Done.

2. **Branche** : `git checkout -b feature/DEVOPS-11-patients-route`
3. **Commit** : `feat(DEVOPS-11): add /api/patients route` (1b6cea7, 7 fichiers, 304 lignes ajoutées)
4. **Push** : `git push origin feature/DEVOPS-11-patients-route`
5. **Pull request** : #5, liée à Jira par la clé DEVOPS-11 présente dans la branche et le commit.
6. **CI sur la PR** : `npm ci`, `npm run lint`, `npm test -- --coverage` → succès à 16 h 04.
7. **Merge** dans `main` à 16 h 05, puis CI sur main : succès.

Le workflow `.github/workflows/ci.yml` (job `build-and-test`) s'exécute sur chaque push ou PR vers `main` : `npm ci --legacy-peer-deps`, `npm run lint`, `npm test -- --coverage`, puis publication de l'artefact `coverage-report`.

Le pipeline sert de garde-fou : si le lint échoue ou si la couverture passe sous les seuils de `vitest.config.js`, la CI échoue et bloque le merge.

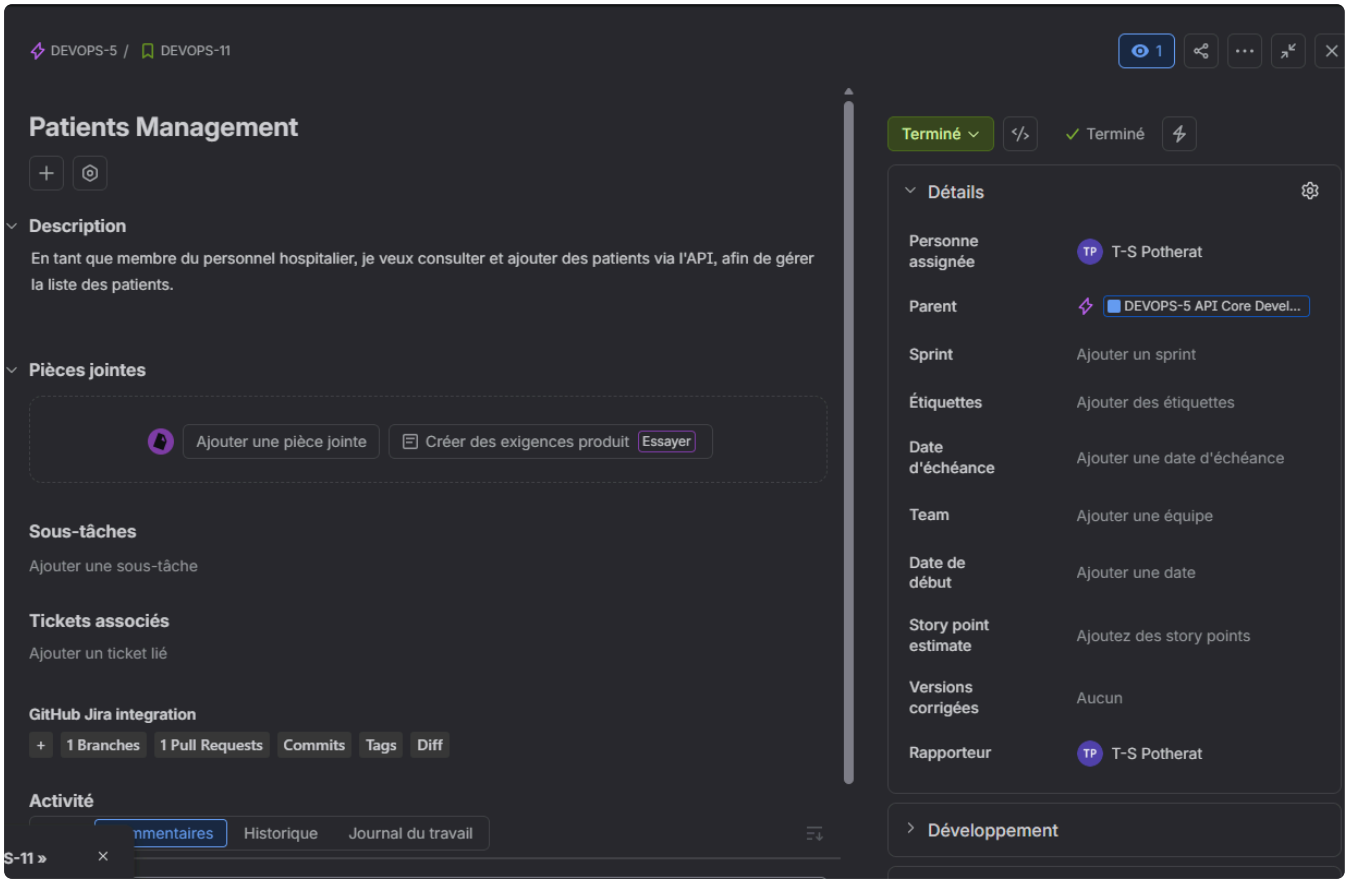


Figure 5 – Story DEVOPS-11 « Patients Management », enfant de l'epic DEVOPS-5, statut Terminé, liée à 1 branche et 1 pull request GitHub.



Figure 6 – Onglet Actions : CI #16 et #17 sur la PR #5 (16 h 04), CI #18 sur `main` après le merge (16 h 05), toutes au vert.

Commit de finition sur `feature/DEVOPS-11-patients-finalize` : `chore(DEVOPS-11): finalize patients route and team Postman collection` (6d3e890). Il fusionne la collection Postman de l'équipe en un seul export, retire un garde inutile dans la route et un `express.json()` dupliqué par un merge, et corrige `specialty` dans `js-practice.js`. Sa pull request lance la CI avant le merge dans `main`.

7. Difficultés et conclusion

Difficulté	Cause	Solution
`req.body` vide sur les POST	Express ne parse pas le JSON par défaut	Ajout de `app.use(express.json())` dans `src/app.js`
Validation trop permissive dans l'exemple du sujet	`!age` accepte `"old"` ou `-3`	Contrôle de type : chaîne non vide, entier positif, 6 cas testés
Requêtes absentes de Postman	Le fichier JSON ouvert dans l'éditeur n'est pas une collection importée	Import du fichier via Collections → Import
Pas de corps JSON dans le Runner	Le Runner affiche les assertions, pas les réponses	Envoi de chaque requête avec Send pour voir statut, temps et corps

La route Patient Management est livrée : GET et POST fonctionnent, sont validés, couverts à 100 % par 10 tests Vitest et vérifiés dans Postman (8 assertions sur 8). Le workflow Jira → branche → PR → CI a fonctionné de bout en bout. Limites : les données ne sont pas persistées, et l'id est calculé à partir de la longueur du tableau, ce qui créerait des doublons si une suppression était ajoutée. Une base de données (MongoDB est déjà dans les dépendances) serait la suite logique.